

数据库系统原理课程大作业报告

城市公共服务请求管理与查询系统数据库设计



学 号 2452769

姓 名 幸可函

专 业 计算机科学与技术

授课老师 李文根

目录

1. 需求分析.....	3
2. 概念设计.....	3
2.1 联系集.....	5
2.2 联系集.....	7
2.3 ER 图	8
3. 逻辑设计.....	8
3.1 基础实体关系.....	8
3.1.1 Agency	8
3.1.2 ComplaintType	9
3.1.3 RequestStatus.....	9
3.1.4 ChannelType.....	9
3.1.5 DataSource.....	10
3.1.6 ImportBatch	10
3.2 核心服务请求关系.....	11
3.2.1 ServiceRequest.....	11
3.3 地点复合属性的分解.....	12
3.3.1 RequestAddress	12
3.3.2 RequestCoordinate.....	13
3.3.3 RequestArea.....	13
3.3.4 CommunityBoard.....	14
3.3.5 RequestCommunityBoard.....	14
3.3.6 RequestParcel.....	15
3.4 特殊场景字段的分解.....	15
3.4.1 ParkRequestInfo.....	15
3.4.2 TaxiRequestInfo.....	16
3.4.3 BridgeHighwayRequestInfo	16
4. 物理设计.....	17
5. 系统实现.....	19
参考文献.....	20

1. 需求分析

本系统面向纽约市公共服务请求管理与查询场景，数据来源于 NYC Open Data 平台发布的“311 Service Requests from 2020 to Present”数据集。该数据集由纽约 311 提供，每行记录一个可指派至特定城市机构的服务请求，包含请求创建与关闭时间、响应机构、问题类型与细分描述、处理状态、解决说明、行政区、经纬度、提交渠道等信息，不包含请求者个人身份信息；数据集每日更新，可作为现实开源数据支撑本系统的数据库设计。本系统以纽约市为具体应用城市，围绕公共服务请求的记录、分类、处理状态、空间位置和响应机构等信息进行建模，旨在将原始的开放数据组织为结构清晰、便于查询和统计的关系数据库。

本系统主要面向三类用户：普通查询用户、城市管理分析人员和系统管理员。普通查询用户关注某一类公共服务请求的基本信息，例如问题类型、发生地点、处理状态和解决说明；城市管理分析人员关注不同机构、不同区域、不同问题类型的请求数量、处理状态分布和处理时长等统计信息，以了解城市公共服务需求与治理压力；系统管理员负责开放数据的导入、清洗、更新和基础数据维护。基于上述用户需求，系统支持按问题类型、响应机构、行政区、时间范围、处理状态和提交渠道等条件进行查询，并支持对请求数量、关闭情况、区域分布和机构响应情况进行统计分析。本系统不涉及请求者身份管理、内部派单流程、工作人员绩效考核等原始开放数据无法直接支撑的业务内容。

2. 概念设计

在 311 Service Requests from 2020 to Present 原始数据集中，一条数据记录包含如下字段：

序号	字段	含义	类型
1	Unique Key	服务请求唯一编号	请求标识
2	Created Date	请求创建时间	时间
3	Closed Date	请求关闭时间	时间，可空
4	Agency	响应机构缩写	机构
5	Agency Name	响应机构全称	机构
6	Problem / Complaint Type	问题/投诉类型	问题分类
7	Problem Detail / Descriptor	问题细分描述	问题分类，可空
8	Additional Details	更细一级的问题描述	问题分类，可空
9	Location Type	地点类型	地点
10	Incident Zip	事件地点邮编	地点，可空

序号	字段	含义	类型
11	Incident Address	事件地址	地点, 可空
12	Street Name	街道名	地点, 可空
13	Cross Street 1	交叉街道 1	地点, 可空
14	Cross Street 2	交叉街道 2	地点, 可空
15	Intersection Street 1	路口街道 1	地点, 可空
16	Intersection Street 2	路口街道 2	地点, 可空
17	Address Type	地址类型	地点
18	City	城市/地点城市名	地点, 可空
19	Landmark	地标	地点, 可空
20	Facility Type	城市设施类型	设施, 可空
21	Status	请求状态	状态
22	Due Date	预期更新时间/截止时间	时间, 可空
23	Resolution Description	处理/解决说明	处理结果, 可空
24	Resolution Action Updated Date	最近处理更新时间	时间, 可空
25	Community Board	社区委员会区域	区域
26	Council District	市议会选区	区域, 可空
27	Police Precinct	警区	区域, 可空
28	BBL	Borough-Block-Lot 地块编号	地块, 可空
29	Borough	行政区	区域
30	X Coordinate (State Plane)	平面坐标 X	坐标, 可空
31	Y Coordinate (State Plane)	平面坐标 Y	坐标, 可空
32	Open Data Channel Type	提交渠道, 如电话、在线、移动端	渠道
33	Park Facility Name	公园设施名称	特殊地点, 可空
34	Park Borough	公园所在行政区	特殊地点, 可空
35	Vehicle Type	TLC/出租车相关车辆类型	特殊问题, 可空
36	Taxi Company Borough	出租车公司行政区	特殊问题, 可空
37	Taxi Pick Up Location	出租车上车地点	特殊问题, 可空
38	Bridge Highway Name	桥梁/高速名称	特殊地点, 可空

序号	字段	含义	类型
39	Bridge Highway Direction	桥梁/高速方向	特殊地点，可空
40	Road Ramp	道路/匝道	特殊地点，可空
41	Bridge Highway Segment	桥梁/高速路段	特殊地点，可空
42	Latitude	纬度	坐标，可空
43	Longitude	经度	坐标，可空
44	Location	经纬度组合点	坐标，可空

由此设计：

2.1 联系集

实体集	实体含义	主码	其余属性	属性来源
ServiceRequest	311 公共服务请求；核心强实体集，一条记录对应一个可区分的服务请求	request_id	created_date, closed_date, due_date, resolution_action_updated_date, resolution_description, problem_detail, additional_details, incident_location, condition_specific_info	request_id 来自 Unique Key；时间字段来自 Created Date, Closed Date, Due Date, Resolution Action Updated Date；解决说明来自 Resolution Description；problem_detail 来自 Problem Detail / Descriptor；additional_details 来自 Additional Details；地点和特殊场景字段来自原表对应字段
Agency	响应机构；负责处理服务请求的城市政府机构	agency_code	agency_name	agency_code 来自 Agency；agency_name 来自 Agency Name

实体集	实体含义	主码	其余属性	属性来源
ComplaintType	问题/投诉类型；服务请求所属的一级问题分类	complaint_type_name	无	complaint_type_name 来自 Problem / Complaint Type；官方说明其为问题层级的第一层，由响应机构定义，且可能随需求变化增加
RequestStatus	请求状态；表示服务请求当前处理状态	status_name	无	status_name 来自 Status
ChannelType	提交渠道；表示请求提交到 311 的方式	channel_name	无	channel_name 来自 Open Data Channel Type，如 Phone、Online、Mobile、Other、Unknown
DataSource	数据来源；记录本系统导入的开放数据集来源	source_id	dataset_name, source_name, update_frequency, source_url	系统元数据；数据集名称、来源 311、每日更新等来自 NYC Open Data 官方说明

其中的复合属性：

复合属性	组成属性	来源字段	说明
incident_location	location_type, address_type, incident_zip, incident_address, street_name, cross_street_1, cross_street_2, intersection_street_1, intersection_street_2, city, borough, community_board, council_district, police_precinct, bbl, landmark, facility_type, latitude, longitude	对应原表 Location Type、Address Type 等字段	描述某条服务请求的发生地点或地理归属，不作为独立实体集

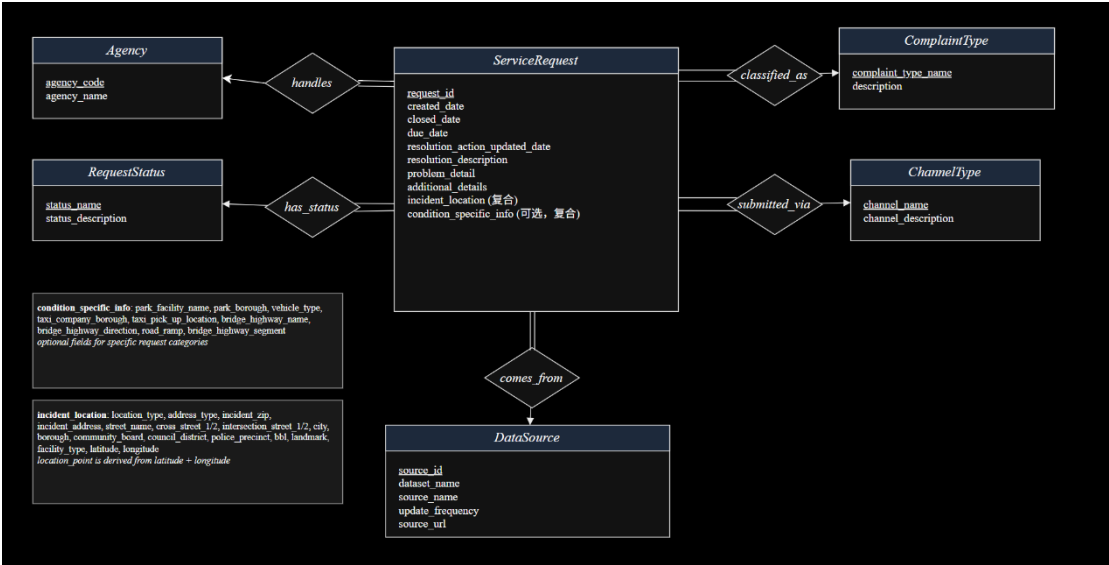
复合属性	组成属性	来源字段	说明
condition_specific_info	park_facility_name, park_borough, vehicle_type, taxi_company_borough, taxi_pick_up_location, bridge_highway_name, bridge_highway_direction, road_ramp, bridge_highway_segment	公园、出租车、 桥梁/高速相关 字段	这些字段只在 特定类型请求 中出现，因此 不作为实体 集，而是 ServiceRequest 的条件性可 空属性

2.2 联系集

联系集	参与实体集	基数约束	参与约束	联系属性	说明
handles	Agency — ServiceRequest	1 : N	ServiceRequest 全部参与；Agency 部分参与	无	一个机构可以处理 多条服务请求；每 条服务请求由一个 响应机构处理
classified_as	ComplaintType — ServiceRequest	1 : N	ServiceRequest 全部参与； ComplaintType 部 分参与	无	一个问题类型可以 对应多条服务请 求；每条服务请求 属于一个一级问题 类型
has_status	RequestStatus — ServiceRequest	1 : N	ServiceRequest 全部参与； RequestStatus 部 分参与	无	一个状态可以对应 多条服务请求；每 条服务请求有一个 状态
submitted_via	ChannelType — ServiceRequest	1 : N	ServiceRequest 全部参与； ChannelType 部分 参与	无	一个提交渠道可以 对应多条请求；每 条请求通过一个渠 道提交
comes_from	DataSource — ServiceRequest	1 : N	ServiceRequest 全部参与；	无	一个数据源提供多 条服务请求；每条

联系集	参与实体集	基数约束	参与约束	联系属性	说明
			DataSource 部分参与		导入的服务请求来自一个数据源

2.3 ER 图



3. 逻辑设计

3.1 基础实体关系

3.1.1 Agency

```

Agency(
    agency_code,    -- 来自原字段 Agency, 表示响应机构缩写
    agency_name,    -- 来自原字段 Agency Name, 表示响应机构全称

```

```

    primary key (agency_code)
)

```

函数依赖: `agency_code → agency_name`

满足 3NF。因为该关系只有一个非平凡函数依赖，且决定属性 `agency_code` 是主码，不存在部分依赖或传递依赖。

3.1.2 ComplaintType

```
ComplaintType(  
    complaint_type_id,    -- 系统生成的问题类型编号  
    complaint_type_name,  -- 来自原字段 Problem / Complaint Type, 表示一级问题分类
```

```
    primary key (complaint_type_id),  
    unique (complaint_type_name)  
)
```

函数依赖: $\text{complaint_type_id} \rightarrow \text{complaint_type_name}$, $\text{complaint_type_name} \rightarrow \text{complaint_type_id}$

满足 3NF。因为 `complaint_type_id` 是主码, `complaint_type_name` 被设置为唯一候选码, 所有非平凡函数依赖的决定属性均为候选码。

3.1.3 RequestStatus

```
RequestStatus(  
    status_id,    -- 系统生成的状态编号  
    status_name,  -- 来自原字段 Status, 表示服务请求状态  
  
    primary key (status_id),  
    unique (status_name)  
)
```

函数依赖: $\text{status_id} \rightarrow \text{status_name}$, $\text{status_name} \rightarrow \text{status_id}$

满足 3NF。因为 `status_id` 是主码, `status_name` 也具有唯一约束, 可作为候选码, 不存在非主属性之间的传递依赖。

3.1.4 ChannelType

```
ChannelType(  
    channel_id,    -- 系统生成的渠道编号  
    channel_name,  -- 来自原字段 Open Data Channel Type, 表示提交渠道  
  
    primary key (channel_id),
```

```
unique (channel_name)
)
```

函数依赖: $\text{channel_id} \rightarrow \text{channel_name}$, $\text{channel_name} \rightarrow \text{channel_id}$

满足 3NF。因为 channel_id 是主码, channel_name 具有唯一约束, 可作为候选码, 不存在部分依赖或传递依赖。

3.1.5 DataSource

```
DataSource(
  source_id,          -- 系统生成的数据源编号
  dataset_name,       -- 数据集名称, 如 311 Service Requests from 2020
                      to Present
  source_name,        -- 数据来源, 如 NYC 311 / NYC Open Data
  update_frequency,   -- 更新频率, 如 Daily
  source_url,         -- 数据集来源 URL

  primary key (source_id),
  unique (source_url)
)
```

函数依赖:

$\text{source_id} \rightarrow \text{dataset_name}, \text{source_name}, \text{update_frequency}, \text{source_url}$

$\text{source_url} \rightarrow \text{source_id}, \text{dataset_name}, \text{source_name}, \text{update_frequency}$

满足 3NF。因为 source_id 是主码, source_url 具有唯一性约束, 也可作为候选码; 关系中没有非主属性决定其他非主属性的传递依赖。

3.1.6 ImportBatch

```
ImportBatch(
  batch_id,          -- 系统生成的导入批次编号
  source_id,         -- 数据来源编号, 引用 DataSource
  import_time,       -- 本批次数据导入时间

  primary key (batch_id),
  foreign key (source_id) references DataSource(source_id)
)
```

函数依赖: $\text{batch_id} \rightarrow \text{source_id}, \text{import_time}$

满足 3NF。因为 batch_id 是主码, 所有非主属性都完全依赖于主码, 不存在非主属性之间的传递依赖。将 import_time 放入 ImportBatch 而不是 DataSource , 是因为同一数据源

可能被多次导入，导入时间属于“导入批次”而不是“数据源”本身。

3.2 核心服务请求关系

3.2.1 ServiceRequest

```
ServiceRequest(  
  request_id,          -- 来自原字段 Unique Key, 表示服务请求唯一编号  
  agency_code,         -- 响应机构编号, 引用 Agency  
  complaint_type_id,   -- 问题类型编号, 引用 ComplaintType  
  status_id,           -- 请求状态编号, 引用 RequestStatus  
  channel_id,          -- 提交渠道编号, 引用 ChannelType  
  batch_id,            -- 导入批次编号, 引用 ImportBatch  
  
  created_date,        -- 来自原字段 Created Date, 表示请求创建时间  
  closed_date,         -- 来自原字段 Closed Date, 表示请求关闭时间, 可  
                        -- 为空  
  due_date,            -- 来自原字段 Due Date, 表示期望处理  
                        -- /更新时间, 可为空  
  resolution_action_updated_date, -- 来自原字段 Resolution Action  
                        -- Updated Date, 表示最后更新时间  
  resolution_description, -- 来自原字段 Resolution  
                        -- Description, 表示处理说明  
  problem_detail,      -- 来自原字段 Problem Detail /  
                        -- Descriptor, 表示二级问题细节, 可为空  
  additional_details,  -- 来自原字段 Additional  
                        -- Details, 表示三级补充细节, 可为空  
  
  primary key (request_id),  
  foreign key (agency_code) references Agency(agency_code),  
  foreign key (complaint_type_id) references  
  ComplaintType(complaint_type_id),  
  foreign key (status_id) references RequestStatus(status_id),  
  foreign key (channel_id) references ChannelType(channel_id),  
  foreign key (batch_id) references ImportBatch(batch_id)  
)
```

函数依赖:

request_id → agency_code, complaint_type_id, status_id, channel_id, batch_id,

created_date, closed_date, due_date, resolution_action_updated_date, resolution_description, problem_detail, additional_details

满足 3NF。因为 request_id 是主码，所有非主属性都完全函数依赖于 request_id。

3.3 地点复合属性的分解

E-R 阶段中，地点信息作为 ServiceRequest 的复合属性 incident_location；进入关系模型后，需要将复合属性展开为原子属性。为了避免 ServiceRequest 过宽，同时降低空值和潜在冗余，分解为以下关系。

3.3.1 RequestAddress

```
RequestAddress(  
    request_id,          -- 服务请求编号，引用 ServiceRequest  
    location_type,       -- 来自原字段 Location Type，表示地点类型  
    address_type,        -- 来自原字段 Address Type，表示地址信息类型  
    incident_zip,        -- 来自原字段 Incident Zip，表示事件地点邮编  
    incident_address,    -- 来自原字段 Incident Address，表示事件地址  
                        门牌/地址信息  
    street_name,         -- 来自原字段 Street Name，表示街道名  
    cross_street_1,      -- 来自原字段 Cross Street 1，表示第一条交叉  
                        街道  
    cross_street_2,      -- 来自原字段 Cross Street 2，表示第二条交叉  
                        街道  
    intersection_street_1, -- 来自原字段 Intersection Street 1，表示第  
                        一条相交街道  
    intersection_street_2, -- 来自原字段 Intersection Street 2，表示第  
                        二条相交街道  
    city,               -- 来自原字段 City，表示事件地点城市  
    landmark,           -- 来自原字段 Landmark，表示地标名称，可为空  
    facility_type,       -- 来自原字段 Facility Type，表示设施类型，可  
                        为空  
  
    primary key (request_id),  
    foreign key (request_id) references ServiceRequest(request_id)  
)
```

函数依赖：

request_id → location_type, address_type, incident_zip, incident_address, street_name,

```
cross_street_1, cross_street_2, intersection_street_1, intersection_street_2, city, landmark,
facility_type
```

满足 3NF。因为 request_id 是主码，所有地址描述属性都完全依赖于该服务请求。这里不额外假设 incident_zip → city，因为当前系统不管理邮编区域对象，city 仅当作服务请求地点的描述属性。

3.3.2 RequestCoordinate

```
RequestCoordinate(
    request_id,    -- 服务请求编号，引用 ServiceRequest
    latitude,      -- 来自原字段 Latitude，表示事件地点纬度
    longitude,     -- 来自原字段 Longitude，表示事件地点经度

    primary key (request_id),
    foreign key (request_id) references ServiceRequest(request_id)
)
```

函数依赖: request_id → latitude, longitude

满足 3NF。因为 request_id 是主码，latitude、longitude 完全依赖于主码，不存在传递依赖。

说明: 不存储原始字段 Location/location_point，因为它可以由 latitude 和 longitude 派生; 也不存储原始字段 x_coordinate、y_coordinate，避免与经纬度表达同一空间点造成冗余。

3.3.3 RequestArea

```
RequestArea(
    request_id,          -- 服务请求编号，引用 ServiceRequest
    borough,             -- 来自原字段 Borough，表示服务请求所在行政区
    city,                -- 来自原字段 City，表示服务请求所在城市
    council_district,    -- 来自原字段 Council District，表示服务请求所在市议会选区
    police_precinct,     -- 来自原字段 Police Precinct，表示服务请求所在警区

    primary key (request_id),
    foreign key (request_id) references ServiceRequest(request_id)
)
```

函数依赖: request_id → borough, city, council_district, police_precinct

满足 3NF。因为 request_id 是主码，borough、city、council_district、police_precinct 都直接描述某条服务请求的地理归属，完全函数依赖于主码。该关系中不再保存 community_board，因此避免了 community_board → borough 可能造成的传递依赖。

3.3.4 CommunityBoard

```
CommunityBoard(  
    community_board, -- 来自原字段 Community Board，表示社区委员会区域  
    borough,         -- 该 Community Board 所属行政区  
  
    primary key (community_board)  
)
```

字段来源说明：community_board 来自原始字段 Community Board；borough 可由 community_board 中包含的行政区信息得到，或由原字段 Borough 辅助确认

例如原始数据中的 Community Board 常见形式类似：05 MANHATTAN、12 QUEENS、01 BROOKLYN。它本身通常已经包含所属 Borough 信息。因此可以认为存在函数依赖：community_board → borough

满足 3NF。因为 community_board 是主码，borough 完全函数依赖于主码；该关系中没有其他非主属性，因此不存在部分依赖或传递依赖。

这里把 CommunityBoard 单独拆出来，是为了避免在每条请求记录中重复存储 community_board, borough，并避免潜在的 request_id → community_board → borough 传递依赖。

3.3.5 RequestCommunityBoard

```
RequestCommunityBoard(  
    request_id,        -- 服务请求编号，引用 ServiceRequest  
    community_board,   -- 社区委员会区域，引用 CommunityBoard  
  
    primary key (request_id),  
    foreign key (request_id) references ServiceRequest(request_id),  
    foreign key (community_board) references  
    CommunityBoard(community_board)  
)
```

函数依赖：request_id → community_board

满足 3NF。因为 request_id 是主码，community_board 完全函数依赖于 request_id，不存在部分依赖或传递依赖。

这里用 request_id 作为主码，是因为一条服务请求最多对应一个 Community Board。如

果某条请求缺少 Community Board 信息,则可以不在 RequestCommunityBoard 中插入对应元组。

3.3.6 RequestParcel

```
RequestParcel(  
    request_id,    -- 服务请求编号, 引用 ServiceRequest  
    bbl,           -- 来自原字段 BBL, 表示 Borough-Block-Lot 地块编号, 可为  
空  
  
    primary key (request_id),  
    foreign key (request_id) references ServiceRequest(request_id)  
)
```

函数依赖: request_id → bbl

满足 3NF。因为 request_id 是主码, bbl 完全依赖于主码。将 bbl 单独拆出, 是为了避免它和 borough 字段混在同一关系中产生潜在依赖 bbl → borough。当前系统不管理地块对象本身, 因此不建立独立的 Parcel 实体关系, 只作为请求的可选地块信息保存。

3.4 特殊场景字段的分解

这些字段只适用于特定类型的服务请求。将它们从 ServiceRequest 中拆出, 避免数据冗余和大量空值。

3.4.1 ParkRequestInfo

```
ParkRequestInfo(  
    request_id,           -- 服务请求编号, 引用 ServiceRequest  
    park_facility_name,   -- 来自原字段 Park Facility Name, 表示公园设施名  
称  
    park_borough,        -- 来自原字段 Park Borough, 表示公园所在行政区  
  
    primary key (request_id),  
    foreign key (request_id) references ServiceRequest(request_id)  
)
```

函数依赖: request_id → park_facility_name, park_borough

满足 3NF。因为 request_id 是主码, 所有非主属性都完全依赖于主码, 不存在部分依赖或传递依赖。该关系只保存与公园设施相关的请求扩展信息, 避免在 ServiceRequest 中产生大量空值。

3.4.2 TaxiRequestInfo

```
TaxiRequestInfo(  
    request_id,          -- 服务请求编号, 引用 ServiceRequest  
    vehicle_type,        -- 来自原字段 Vehicle Type, 表示车辆类型  
    taxi_company_borough, -- 来自原字段 Taxi Company Borough, 表示出租  
                           车公司所在行政区  
    taxi_pick_up_location, -- 来自原字段 Taxi Pick Up Location, 表示出  
                           租车上车地点  
  
    primary key (request_id),  
    foreign key (request_id) references ServiceRequest(request_id)  
)
```

函数依赖: $\text{request_id} \rightarrow \text{vehicle_type}, \text{taxi_company_borough}, \text{taxi_pick_up_location}$

满足 3NF。因为 request_id 是主码, 所有出租车相关属性都完全依赖于该服务请求, 不存在非主属性之间的传递依赖。该关系仅用于出租车相关请求。

3.4.3 BridgeHighwayRequestInfo

```
BridgeHighwayRequestInfo(  
    request_id,          -- 服务请求编号, 引用 ServiceRequest  
    bridge_highway_name,  -- 来自原字段 Bridge Highway Name, 表示桥  
                           梁/高速名称  
    bridge_highway_direction, -- 来自原字段 Bridge Highway Direction,  
                           表示方向  
    road_ramp,           -- 来自原字段 Road Ramp, 表示道路/匝道信息  
    bridge_highway_segment, -- 来自原字段 Bridge Highway Segment, 表  
                           示桥梁/高速路段  
  
    primary key (request_id),  
    foreign key (request_id) references ServiceRequest(request_id)  
)
```

函数依赖:

$\text{request_id} \rightarrow$

```
    bridge_highway_name,  
    bridge_highway_direction,  
    road_ramp,
```

bridge_highway_segment

满足 3NF。因为 request_id 是主码，所有桥梁/高速相关属性都完全依赖于该服务请求，不存在部分依赖或传递依赖。该关系仅用于桥梁/高速相关请求，避免在 ServiceRequest 中产生大量空值。

4. 物理设计

本系统的核心数据表为 ServiceRequest，其数据量预计远大于其他字典表和扩展表。因此，物理设计的重点是为服务请求的高频查询、连接操作和统计分析建立合适的索引结构。本文主要采用 B+ 树索引。B+ 树索引既支持等值查询，也支持范围查询，适合本系统中按时间、机构、问题类型、状态和区域进行筛选的查询场景。

首先，各关系的主码自动建立主码索引。例如：

```
ServiceRequest(request_id)
Agency(agency_code)
ComplaintType(complaint_type_id)
RequestStatus(status_id)
ChannelType(channel_id)
DataSource(source_id)
ImportBatch(batch_id)
CommunityBoard(community_board)
```

这些索引用于保证实体唯一性，并加速通过主码访问元组。

然后，在 ServiceRequest 的外码字段上建立 B+ 树索引，以提高连接查询和条件筛选效率：

```
create index idx_request_agency
on ServiceRequest using btree (agency_code);

create index idx_request_complaint_type
on ServiceRequest using btree (complaint_type_id);

create index idx_request_status
on ServiceRequest using btree (status_id);

create index idx_request_channel
on ServiceRequest using btree (channel_id);

create index idx_request_batch
on ServiceRequest using btree (batch_id);
```

这些索引主要服务于如下查询：按响应机构统计请求数量、按问题类型查询服务请求、按状态筛选未关闭请求、按提交渠道分析请求来源等。

由于系统需要按时间范围查询服务请求，因此在 `created_date` 上建立 B+ 树索引：

```
create index idx_request_created_date
on ServiceRequest using btree (created_date);
```

该索引可用于支持如下查询：

- 查询某一时间段内的服务请求；
- 统计某月、某季度、某年不同类型请求数量；
- 分析请求量随时间变化的趋势。

对于常见的组合查询，可进一步建立组合索引：

```
create index idx_request_type_date
on ServiceRequest using btree (complaint_type_id, created_date);
```

```
create index idx_request_agency_date
on ServiceRequest using btree (agency_code, created_date);
```

```
create index idx_request_status_date
on ServiceRequest using btree (status_id, created_date);
```

其中，(complaint_type_id, created_date) 可支持“某类问题在某时间段内的请求查询”；(agency_code, created_date) 可支持“某机构在某时间段内处理的请求查询”；(status_id, created_date) 可支持“某时间段内不同状态请求查询”。

地点和区域信息被分解到 RequestArea、CommunityBoard 和 RequestCommunityBoard 等关系中。为支持区域统计，可建立如下索引：

```
create index idx_request_area_borough
on RequestArea using btree (borough);
```

```
create index idx_request_area_city
on RequestArea using btree (city);
```

```
create index idx_request_area_council
on RequestArea using btree (council_district);
```

```
create index idx_request_area_precinct
on RequestArea using btree (police_precinct);
```

```
create index idx_request_cb_board
on RequestCommunityBoard using btree (community_board);
```

其中，borough、city、council_district、police_precinct 可用于按行政区、城市、市议会

选区和警区统计服务请求；community_board 索引用于按社区委员会查询相关请求。

综上，本系统物理设计主要采用 B+ 树索引，重点索引主码、外码、时间字段和高频统计字段。该设计能够支持按请求编号、机构、问题类型、状态、提交渠道、时间范围和区域维度进行高效查询，同时避免过度索引带来的存储和更新开销。

5. 系统实现

本系统可基于关系型数据库管理系统实现，例如使用 MySQL 或 PostgreSQL 存储清洗后的 311 服务请求数据。系统通过数据导入模块从 NYC Open Data 获取开放数据，对原始字段进行清洗、类型转换和去重处理，再按本报告设计的关系模式写入数据库。由于数据集每日更新，且每条记录包含问题类型、响应机构、地理位置等不同信息，系统需采用定期批量导入的方式维护数据。

应用层可设计为 Web 查询与统计系统，主要功能包括：按请求编号查询详情；按时间范围、响应机构、问题类型、处理状态、提交渠道和区域条件筛选记录；并生成请求数量、关闭情况、机构响应情况和区域分布等统计结果。后端通过 SQL 查询和索引优化提高检索效率，前端以表格、筛选器和简单统计图展示结果。本阶段主要完成数据库设计，系统具体实现待下学期课程设计完成。

参考文献

[1] NYC Open Data. 311 Service Requests from 2020 to Present[EB/OL]. [311 Service Requests from 2020 to Present | NYC Open Data](#)